



Test End-to-End avec Selenium



AGL



Test de bout en bout



- ▶ Test de l'application dans son ensemble
- ▶ Avec un point de vue utilisateur
- ▶ En utilisant des scénarios d'utilisation réalistes
- ▶ Dernières étapes de test (après le test unitaire et le test d'intégration)



Bénéfices



- ▶ valuer l'adéquation entre les besoins initiaux et le produit fini
- ▶ permet d'évaluer la bonne intégration complète des composants
- ▶ permet de valider les principaux scénarios d'utilisation



Processus de mise en place



- 1 Elaboration de scénarios de tests depuis les interfaces
- 2 Mise en place des environnements de test



Difficultés



- ▶ C'est long !
 - ◆ se souvenir de la pyramide de test ...
 - ◆ il y a beaucoup de scénarios possibles, beaucoup de combinaisons de sous-scénarios
- ▶ C'est fastidieux
 - ◆ plusieurs browsers à tester par exemple
 - ◆ oracles compliqués à mettre en place
 - ◆ environnement de test à mettre en place
- ▶ C'est compliqué de mettre en place les "bons" scénarios



- ▶ Beaucoup d'outils permettant l'automatisation des tests depuis l'UI
- ▶ Notamment depuis une interface web
 - ◆ Selenium
 - ◆ Cypress
 - ◆ Katalon studio
 - ◆ ...



Les tests E2E ...



- ▶ ne dispensent pas (du tout !) de faire des tests unitaires / d'intégration / d'UI
- ▶ sont coûteux à mettre en place, longs à exécuter
- ▶ doivent co-évoluer avec le SUT (comme les autres tests)
- ▶ peuvent être partiellement remplacés par des tests sous-cutanés ...
 - ◆ <https://martinfowler.com/bliki/SubcutaneousTest.html>



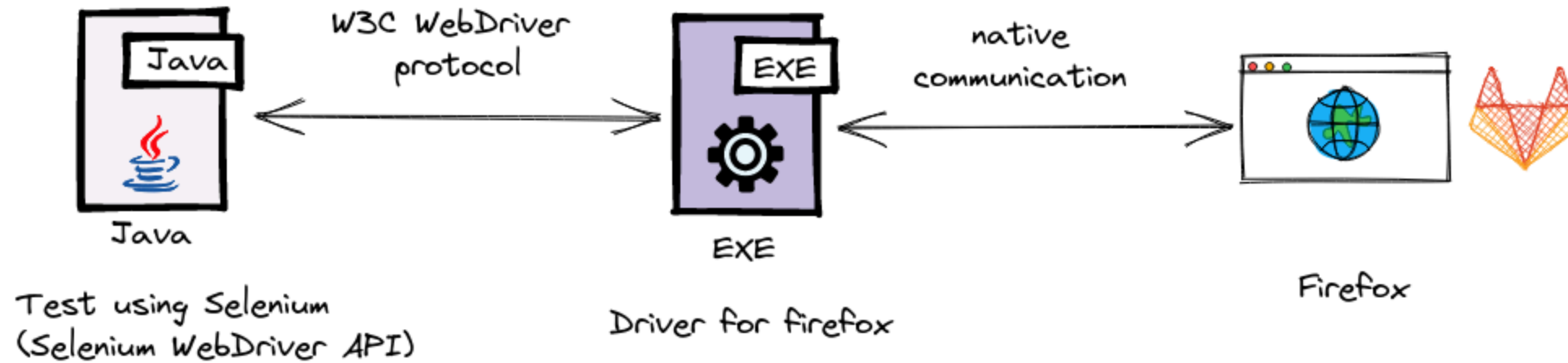
- ▶ Pour la petite histoire ...
 - ◆ Le sélénium est le contre-poison au mercure
- ▶ Selenium WebDriver
 - ◆ pilote les navigateurs (classiques)
- ▶ Utilisation du DOM



Exemple de test Selenium



- ▶ demander au navigateur de récupérer la page à une certaine adresse (get),
- ▶ demander au navigateur de récupérer dans la page reçue les différents éléments d'un formulaire,
- ▶ demander au navigateur de remplir ces éléments avec des valeurs (données de test),
- ▶ demander au navigateur de cliquer sur le bouton de validation du formulaire,
- ▶ puis vérifier que tout s'est bien passé (pas d'erreur, nouvelle page reçue contenant les bonnes valeurs, etc).





Trouver des éléments dans le DOM, locators



- ▶ Locator : moyen d'identifier une/des élément(s) dans une page ; transmis aux méthodes de recherche.
- ▶ Stratégies fournies :
 - ◆ class name. localise les éléments ayant le nom de classe fourni.
 - ◆ css selector. localise les éléments matchant le sélecteur css, syntaxe : `css = #id` ou `css =[attribute=value]` .
 - ◆ id : localisation selon l'attribute ID
 - ◆ name : localisation selon l'attribute NAME
 - ◆ link text
 - ◆ partial link
 - ◆ tag name
 - ◆ xpath



Locators, exemple



```
<html>
  <body>
    <style>
      .information {
        background-color: white;
        color: black;
        padding: 10px;
      }
    </style>
    <h2>Contact Selenium</h2>
    <form action="/action_page.php">
      <input type="radio" name="gender" value="m" />Male &nbsp;  
      <input type="radio" name="gender" value="f" />Female <br>
      <br>
      <label for="fname">First name:</label><br>
      <input class="information" type="text" id="fname" name="fname" value="Jane">
      <br><br>
      <label for="lname">Last name:</label><br>
      <input class="information" type="text" id="lname" name="lname" value="Doe">
      <br><br>
      <label for="newsletter">Newsletter:</label>
      <input type="checkbox" name="newsletter" value="1" />
      <br><br>
      <input type="submit" value="Submit">
    </form>
    <p>To know more about Selenium, visit the official page
      <a href = "www.selenium.dev">Selenium Official Page</a>
    </p>
  </body>
</html>
```



Locators, exemple (Java)



```
WebDriver driver = new ChromeDriver();  
// boîtes de texte de nom et prénom  
driver.findElements(By.className("information"));  
// boîte de texte de prénom  
driver.findElements(By.cssSelector("#fname"));  
// boîte de texte de nom  
driver.findElement(By.id("lname"));  
// la case à cocher  
driver.findElements(By.name("newsletter"));  
// le lien (<a>)  
driver.findElement(By.linkText("Selenium Official Page"));  
// le même !  
driver.findElements(By.partialLinkText("Official Page"));  
// encore le même  
driver.findElement(By.tagName("a"));  
// chemin relatif ici,  
//localise le bouton radio female ;  
//chemin absolu possible depuis la racine du doc, ex : /html/form/input[1]  
driver.findElement(By.xpath("//input[@value='f']"));
```

- ▶ findElement → WebElement = 1 élément (le premier rencontré qui matche le locator)
- ▶ findElements → List = les éléments qui matchent le locator

```
<ol id="vegetables">
  <li class="potatoes">...
  <li class="onions">...
  <li class="tomatoes"><span>Tomato is a Vegetable</span>...
</ol>
<ul id="fruits">
  <li class="bananas">...
  <li class="apples">...
  <li class="tomatoes"><span>Tomato is a Fruit</span>...
</ul>
```

```
WebElement vegetable =  
    driver.findElement(By.className("tomatoes")); // tomate légume  
WebElement fruits =  
    driver.findElement(By.id("fruits"));  
WebElement fruit =  
    fruits.findElement(By.className("tomatoes")); // tomate fruit  
WebElement fruit = // tomate fruit  
    driver.findElement(By.cssSelector("#fruits .tomatoes"));  
  
List<WebElement> plants =  
    driver.findElements(By.tagName("li")); // fruits et légumes  
for (WebElement element : plants) {  
    System.out.println(element.getText());  
}
```



tests E2E et Selenium



- ▶ Tests intégrables dans une chaîne CI/CD (mais attention à leur lenteur)
- ▶ Attention aux faux positifs (problèmes de configuration de l'environnement de test)
- ▶ Attention aux faux négatifs (oracle forcément faible)